Proposal on

COMP 4330SEF Advanced Programming and Ai Algorithms

Autumn Term 2026

P01 Group 2

Lam Yu Kwan	13487539
Tsang Yik Long	13487503
To Chung Hei	13493141
Siu Jyut	13483577
RAWAT Aaditya	13231466

1. Topic

Evaluating CPU Inference Performance of scikit-learn and ONNX Runtime

2. *Introduction*

Machine learning model development primarily utilizes scikit-learn due to its concise API and suitability.
However, when deploying models to production environments, scikit-learn's Python execution often becomes a limitation in terms of latency and resource consumption.
The ONNX Runtime, an engine optimized for inference, which can convert scikit-learn models to ONNX format to improve execution efficiency.

Generally, the conversion process is lossy. Besides differences in floating-point precision, additional processing of the output format can also increase inference costs, which may lead to numerical differences in prediction results.

Our purpose is to find out the difference in performance and prediction results when using the same machine learning model to perform inference between scikit-learn and ONNX Runtime, focuses on comparing the execution of the same model.

3. *Literature Review*

Using scikit-learn with ONNX is a clear way to train a machine learning model in Python and then run it in many other places. Scikit-learn is a popular Python library for building classic models, such as linear regression, logistic regression, decision trees, random forests, and support vector machines. ONNX is a standard file format for trained models. It lets one model file work with different tools and programming languages. The usual steps are simple. First, you train your model or full pipeline with scikit-learn. For example, you might use a StandardScaler to scale numbers and a RandomForestClassifier to classify data. Second, you use a tool called skl2onnx to convert the trained model into an ONNX file. During this step, you must tell the converter the input shape and type, such as a float tensor with any batch size and a fixed number of features. Third, you load the ONNX file with ONNX Runtime or another ONNX-compatible engine. You no longer need scikit-learn to make predictions. This separation is helpful because scikit-learn is great for training, while ONNX is great for portable and fast inference.

One important use is lightweight web services. A lightweight web service is a small API that answers prediction requests over HTTP or gRPC. It might be built with FastAPI, Flask, Uvicorn, or Gunicorn. It might run as a serverless function on AWS Lambda, or as a small Docker container on Google Cloud Run. It loads the ONNX file when it starts and exposes an endpoint like /predict. Because ONNX Runtime is small and fast, the service can score JSON input in milliseconds without installing the whole scikit-learn library. Another important use is edge services. Edge services run close to where data is created, not in a faraway cloud data centre.

Examples include Azure SQL Edge, AWS IoT Greengrass, Raspberry Pi devices, NVIDIA Jetson boards, Windows ML on Windows devices, and ONNX Runtime Mobile on phones. These services help with IoT monitoring, predictive maintenance, anomaly detection, and privacy-sensitive tasks. They save bandwidth, avoid cloud round trips, and can keep working offline.

Real applications include Google BigQuery ML, where you import an ONNX model and call ML.PREDICT in SQL; Azure SQL Edge, which runs ONNX models inside a small database; Windows ML, which runs ONNX models on Windows hardware(ML.NET), which lets .NET apps use ONNX models; and custom FastAPI, Flask, or Triton services that use ONNX Runtime. In short, scikit-learn trains the model, and ONNX runs it in many places, including small web services and edge devices, with less code, fewer dependencies, faster predictions, and very easy to use.

4. *Methodology*

Design a Random Forest Classifier model with fixed hyperparameters with Breast Cancer Wisconsin data as a controllable experiment dataset. We planned to process in a cloud VM with CPU only.
The testing method will be comparing ZipMap by observing the difference between ZipMap with default ONNX and when close ZipMap, to see how scikit-learn and ONNX changes when batch number changes (1, 16, 64, 256).
We are going to compare three following settings, scikit-learn predict_proba(), ONNX Runtime with ZipMap enabled, and ONNX Runtime with ZipMap disabled.
Inference time, throughput, label agreement and probability metrics will be included to see the differences.

5. *Work allocation*

Group leader: Lam Yu Kwan
Works:
Siu Juut: Data Preparation and Model Training, Power Point
To Chung Hei: Model Conversion and Basic Inference
RAWAT Aadidya: Performance Benchmark Implementation, literature review
Lam Yu Kwan: Prediction Consistency Verification, Methodology
Tsang Yik Long: ZipMap Improvement Analysis and Overall Report Coordination, Methodology

6. *References*

Cho, H., & Li, M. (2018). Treelite: Toolbox for decision tree deployment. In Proceedings of the 2018 Conference on Systems and Machine Learning (SysML).mlsys
Jin, T., Bercea, G.-T., Le, T. D., Chen, T., Su, G., Imai, H., Negishi, Y., Leu, A., O'Brien, K., Kawachiya, K., & Eichenberger, A. E. (2020). Compiling ONNX neural network models using MLIR [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2008.08272arxiv
Nakandala, S., Saur, K., Yu, G.-I., Karanasos, K., Curino, C., Weimer, M., & Interlandi, M. (2020). A tensor compiler for unified machine learning prediction serving. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20) (pp. 899–917). USENIX Association.usenix
Prasad, A., Rajendra, S., Rajan, K., Govindarajan, R., & Bondhugula, U. (2022). Treebeard: An optimizing compiler for decision tree based ML inference. In 2022